

QVerse — Week 7: The Practical Gate Toolbox

Theory Booklet | Introduction to Quantum Computing & Programming

QVerse

2026

How this booklet fits the course

This theory booklet is designed to be read alongside `07_gate_toolbox_rotations_control.ipynb`. It contains the conceptual and mathematical material that should be explained during Week 7; the notebook then turns those ideas into experiments, circuit construction, and exercises.

Learning objectives

- Use S, T, Rx, Ry, and Rz.
- Use controlled operations.
- Build SWAP from CX gates.
- Reason about gate composition, depth, and identities.

Recommended workflow

1. Read the relevant theory section.
2. Predict the result of the corresponding notebook experiment.
3. Run the notebook.
4. Explain any difference between the prediction and the output.
5. Complete the core exercises without treating the solution notebook as a substitute for reasoning.

1. From a small gate set to a practical toolbox

The first six weeks used X , Z , H , and CNOT. Real quantum programming also requires continuous rotations, controlled operations, and awareness of circuit cost.

The goals of this week are to connect gate matrices with Bloch-sphere rotations and to begin thinking like a compiler: two circuits may be mathematically equivalent but have different depth and hardware cost.

2. Rotation gates

The standard one-qubit rotation gates are

$$R_x(\theta) = e^{-i\theta X/2},$$

$$R_y(\theta) = e^{-i\theta Y/2},$$

$$R_z(\theta) = e^{-i\theta Z/2}.$$

Explicitly,

$$R_x(\theta) = \begin{pmatrix} \cos(\theta/2) & -i \sin(\theta/2) \\ -i \sin(\theta/2) & \cos(\theta/2) \end{pmatrix},$$

$$R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix},$$

$$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}.$$

They correspond to rotations of the Bloch vector around the x , y , and z axes.

3. $R_y(\theta)$ on $|0\rangle$

Applying $R_y(\theta)$ to $|0\rangle$ gives

$$R_y(\theta)|0\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)|1\rangle.$$

Therefore,

$$P(1) = \sin^2\left(\frac{\theta}{2}\right).$$

Special cases:

$$\theta = 0 : \quad |0\rangle,$$

$$\theta = \frac{\pi}{2} : \quad \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle,$$

$$\theta = \pi : \quad |1\rangle.$$

This makes R_y useful for constructing tunable ansätze in Week 13.

4. S and T as fixed phase rotations

The gates

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$$

are related to z -axis rotations up to global phase.

The exact global phase does not affect ordinary circuit outcomes, so fixed phase gates can often be understood as particular R_z -type rotations.

The T gate is especially important in fault-tolerant quantum computing because it is a non-Clifford gate used to achieve universal computation together with Clifford operations.

That advanced topic is outside the scope of the notebook, but it explains why T appears so often in quantum software.

5. Controlled operations

A controlled- U gate applies U to a target only when a control qubit is in the active computational-basis state.

CNOT is the controlled version of X .

Controlled operations are crucial for:

- entanglement;
- conditional logic within coherent quantum circuits;
- phase estimation;
- QFT constructions;
- oracles;
- many communication protocols.

Unlike classical `if` statements, a controlled gate can act coherently when the control qubit is in superposition.

6. CNOT does not clone arbitrary states

For computational-basis inputs,

$$|0\rangle|0\rangle \xrightarrow{\text{CX}} |0\rangle|0\rangle,$$

$$|1\rangle|0\rangle \xrightarrow{\text{CX}} |1\rangle|1\rangle.$$

This can look like copying.

But for an arbitrary unknown state

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

the same circuit produces

$$\alpha|00\rangle + \beta|11\rangle,$$

not

$$|\psi\rangle|\psi\rangle.$$

The desired cloned state would be

$$(\alpha|0\rangle + \beta|1\rangle)^2,$$

which generally contains additional cross terms.

This illustrates the **no-cloning theorem**: there is no universal unitary that can perfectly copy an arbitrary unknown quantum state.

7. SWAP gate from three CNOTs

The SWAP operation exchanges two qubit states:

$$|\psi\rangle|\phi\rangle \longrightarrow |\phi\rangle|\psi\rangle.$$

A standard decomposition is

$$\text{SWAP}(a, b) = \text{CX}_{a \rightarrow b} \text{CX}_{b \rightarrow a} \text{CX}_{a \rightarrow b}.$$

The notebook verifies this on all computational-basis inputs.

This decomposition matters because some hardware may not provide a native SWAP gate. Instead, routing is implemented using native two-qubit operations.

8. Circuit equivalence

Two circuits may implement the same unitary transformation even if their gate sequences look different.

Examples include

$$HZH = X,$$

$$HXH = Z.$$

Sometimes circuits are equal only up to a global phase. For quantum-state predictions, that is still physical equivalence.

Checking equivalence using an operator representation is stronger than comparing one histogram, because one histogram tests only one input and one measurement basis.

9. Gate count and circuit depth

Gate count is the total number of operations.

Circuit depth is the number of sequential layers after operations that can run in parallel are grouped together.

Depth matters because real qubits remain coherent for only a limited time. A deeper circuit generally exposes the state to more errors.

Two-qubit gates are often especially costly. Therefore useful hardware metrics include:

- total depth;
- number of two-qubit gates;
- number of measurements;
- critical path length;
- compatibility with the device connectivity graph.

Week 14 will revisit these metrics during transpilation.

10. Parallel operations

If two gates act on disjoint qubits, they may occupy the same circuit layer.

For example, applying H to q_0 and X to q_1 can conceptually happen in parallel.

Operations that depend on the same qubit generally cannot be parallelized arbitrarily.

This distinction means that gate count and depth are not the same resource.

11. Universal gate sets

A gate set is universal if arbitrary unitary operations can be approximated to any desired precision using gates from that set.

Different platforms use different native gates. At the algorithm level, software exposes convenient abstract gates; the transpiler later converts them into hardware-compatible instructions.

Students do not need to memorize a specific universal gate set for this course, but they should understand why a circuit drawn with high-level gates may be decomposed before hardware execution.

12. Common misconceptions

“CNOT copies a qubit”

It copies computational-basis information only. On arbitrary superpositions it can create entanglement instead.

“The circuit with fewer gates is always better”

Not necessarily. Parallelism, native gate fidelities, connectivity, and compiler choices matter.

“A rotation angle is the measurement probability”

For R_y , probability depends on $\sin^2(\theta/2)$, not linearly on θ .

“Equivalent matrices must be numerically identical”

Global phase can make two physically equivalent unitaries differ element by element.

13. What to know before the notebook

You should be able to:

- interpret R_x , R_y , and R_z as Bloch-sphere rotations;
- derive $P(1) = \sin^2(\theta/2)$ for $R_y(\theta)|0\rangle$;
- explain controlled operations;
- explain why CNOT does not violate no-cloning;
- describe the three-CNOT SWAP;
- distinguish gate count from circuit depth.

Suggested references

- IBM Quantum Learning, *Basics of Quantum Information* and current Qiskit learning material: <https://quantum.cloud.ibm.com/learning/en/courses>
- IBM Quantum Documentation and Qiskit guides: <https://quantum.cloud.ibm.com/docs>
- Quantum Rings, *Quantum 101*: <https://www.quantumrings.com/course/quantum101>
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press.

The software-facing details in the practical notebook should be treated as the canonical implementation for this course. The purpose of this booklet is to explain the physics, mathematics, and algorithmic reasoning behind that implementation.